

Chapitre 4 : Fonctions et modularité

Oscar Junior WAGOUSSI

ENEAM

February 26, 2025

Objectifs du Chapitre 4

- ▶ Comprendre la notion de fonction et son utilité en programmation.
- ▶ Savoir déclarer et utiliser une fonction en Python (`def`).
- ▶ Gérer les paramètres et valeurs de retour.
- ▶ Appréhender la réutilisation du code grâce aux fonctions.
- ▶ Connaître les bonnes pratiques de conception de fonctions.

Qu'est-ce qu'une fonction ?

- ▶ Une fonction est un bloc de code autonome qui effectue une tâche précise.
- ▶ Elle permet d'organiser le code en modules réutilisables et d'être appelée plusieurs fois.
- ▶ Les fonctions améliorent la lisibilité et la maintenance du code.

Avantages :

- ▶ Réduction de la redondance.
- ▶ Facilite le développement et la déburrage.
- ▶ Permet de décomposer des problèmes complexes en sous-tâches.

Syntaxe de déclaration d'une fonction

Listing 1: Déclaration d'une fonction

```
1 def addition(a: int, b: int) -> int:  
2     return a + b  
3
```

Ici, `def` indique le début d'une fonction. Les annotations de type (`a: int, b: int`) `-> int` indiquent que les arguments `a` et `b` doivent être des entiers et que la fonction renvoie un entier.

Paramètres et valeurs de retour

- ▶ Une fonction peut prendre des paramètres (arguments) pour travailler sur des données fournies.
- ▶ Elle peut retourner une valeur avec le mot-clé `return`.
- ▶ Les annotations de type (type hints) permettent de documenter le type attendu des arguments et de la valeur de retour.

Exemple de fonction avec paramètres et valeur de retour :

- ▶ Voir la fonction `addition` ci-dessus.

Réutilisation du code avec les fonctions

- ▶ Les fonctions permettent de réutiliser le même bloc de code plusieurs fois sans le recopier.
- ▶ Cela facilite la maintenance, car une modification dans la fonction se reflétera partout où elle est appelée.
- ▶ Exemple : Calculer l'addition de deux nombres dans différents contextes.

Bonnes pratiques de conception de fonctions

- ▶ **Nommer clairement** la fonction pour refléter son comportement (ex. `calculer_moyenne`).
- ▶ Limiter la taille de la fonction pour qu'elle effectue une seule tâche.
- ▶ Documenter la fonction avec des docstrings (description, paramètres, valeur de retour).
- ▶ Utiliser des annotations de type pour une meilleure lisibilité et pour aider les outils de vérification statique.

Définition et appel d'une fonction

Exemple : Une fonction sans paramètre

Listing 2: Une fonction sans paramètre

```
1 def indique_mon_age():  
2     return 30  
3  
4 # Appel de la fonction  
5 print(indique_mon_age())  
6
```

Explication :

- ▶ La fonction est définie avec le mot-clé `def`.
- ▶ Elle ne prend aucun paramètre et retourne toujours la valeur 30.
- ▶ L'appel `indique_mon_age()` affiche 30.

Fonction avec paramètres obligatoires

Exemple : Addition avec paramètres

Listing 3: Addition avec paramètres

```
1 def augmente_moi(a, b):  
2     return 30 + a + b  
3  
4 # Appel avec des arguments  
5 print(augmente_moi(5, 10))  
6
```

Explication :

- ▶ La fonction prend deux paramètres a et b.
- ▶ Ces paramètres doivent être fournis à l'appel.
- ▶ `augmente_moi(5, 10)` retourne $30 + 5 + 10 = 45$.

Utilisation des paramètres variables (*args)

Exemple : Addition de plusieurs valeurs

Listing 4: Addition de plusieurs valeurs

```
1 def somme_variable(*param):  
2     return sum(param)  
3  
4 # Appel avec plusieurs arguments  
5 print(somme_variable(5, 10, 15))  
6
```

Explication :

- ▶ L'astérisque * permet de passer un nombre variable d'arguments.
- ▶ Tous les arguments sont traités comme une liste.
- ▶ `somme_variable(5, 10, 15)` retourne $5 + 10 + 15 = 30$.

Exemple pratique : Fonction de conversion

Listing 5: Conversion de Celsius en Fahrenheit

```
1 def celsius_to_fahrenheit(celsius: float) -> float:
2     """
3     Convertit une température de Celsius en Fahrenheit.
4     Paramètre:
5         celsius (float): Température en Celsius.
6     Retour:
7         float: Température en Fahrenheit.
8     """
9     return (9/5) * celsius + 32
10
11 # Exemple d'utilisation
12 temp_c = 25.0
13 temp_f = celsius_to_fahrenheit(temp_c)
14 print("25°C en Fahrenheit =", temp_f)
15
```

Les fonctions natives en Python (1/3)

Définition : Une fonction native en Python est une fonction qui fait partie de la bibliothèque standard et est déjà préinstallée.

Fonction	Rôle	Exemple
<code>abs(x)</code>	Valeur absolue d'un nombre	<code>abs(-10) → 10</code>
<code>all(iterable)</code>	Vrai si tous les éléments sont vrais	<code>all([True, False]) → False</code>
<code>any(iterable)</code>	Vrai si au moins un élément est vrai	<code>any([False, True]) → True</code>
<code>bin(x)</code>	Convertit un entier en binaire	<code>bin(10) → '0b1010'</code>
<code>callable(obj)</code>	Vérifie si un objet est callable	<code>callable(len) → True</code>

Les fonctions natives en Python (2/3)

Fonction	Rôle	Exemple
<code>len(s)</code>	Renvoie la longueur d'un objet	<code>len("Python")</code> → 6
<code>max()</code> / <code>min()</code>	Renvoie le max/min d'une liste	<code>max([1,5,3])</code> → 5
<code>round(x)</code>	Arrondit un nombre	<code>round(3.1415,2)</code> → 3.14
<code>str.upper()</code>	Convertit une chaîne en majuscules	<code>"hello".upper()</code> → "HELLO"
<code>random()</code>	Nombre aléatoire entre 0 et 1	<code>random.random()</code> → 0.6742

Les fonctions natives en Python (3/3)

Fonction	Rôle	Exemple
<code>str.capitalize()</code>	Met la première lettre en majuscule	<code>"python".capitalize()</code> → <code>"Python"</code>
<code>str.find(substr)</code>	Trouve la position d'une sous-chaîne	<code>"hello".find("e")</code> → <code>1</code>
<code>str.count(substr)</code>	Compte les occurrences d'une sous-chaîne	<code>"banana".count("a")</code> → <code>3</code>
<code>hex(x)</code>	Convertit un entier en hexadécimal	<code>hex(255)</code> → <code>'0xff'</code>
<code>choice(seq)</code>	Retourne un élément aléatoire	<code>choice([1,2,3])</code> → <code>2</code>

Documentation complète

Pour plus de détails, consultez la documentation officielle de Python :

► <https://docs.python.org/3/library/functions.html>

Exercices pratiques

- ▶ **Exercice 1** : Écrire une fonction `carre(n)` qui retourne le carré d'un nombre `n`.
- ▶ **Exercice 2** : Écrire une fonction `est_pair(n)` qui renvoie `True` si `n` est pair, sinon `False`.
- ▶ **Exercice 3** : Créer une fonction `afficher_infos(nom, age)` qui prend en paramètres un nom et un âge, et affiche un message de bienvenue personnalisé.

Conclusion du Chapitre 4

- ▶ Les fonctions permettent de modulariser le code et de le rendre réutilisable.
- ▶ La déclaration, les paramètres, et la valeur de retour constituent les bases de leur utilisation.
- ▶ Les bonnes pratiques (nommage, documentation, annotations de type) facilitent la maintenance et la lisibilité du code.

Pratiquez en créant vos propres fonctions pour automatiser des tâches simples !